

BUSINESS REQUIREMENTS DOCUMENT

WinCapture

BRD v1.0 | 09 September 2026

1. Document Control

Item	Details
Document	Business Requirements Document (BRD)
Use Case	WinCapture – File Upload and Gallery Platform
Version	1.0
Status	Draft / For Mentor Review
Date	09 September 2026
Business Owner	Sree Navya K L N
Stakeholders	WinWire Technologies – Part of NTT DATA

2. Executive Summary

The proposed WinCapture system is a backend web API for secure file upload, storage, retrieval, replacement, download, and deletion. The application supports two roles, User and Admin, and uses JWT-based authentication with role-based authorization to control access to file resources.

The system is implemented with C# and ASP.NET Core Web API on .NET 10. Entity Framework Core is used for database access to Microsoft SQL Server, while Azure Blob Storage is used to store the actual uploaded file content. SQL Server keeps user and file metadata, including the original file name, stored file name, content type, size, uploader, and upload time.

The project also includes centralized exception handling, input validation, Swagger/OpenAPI support, BCrypt password hashing, and the repository/service pattern. The implementation is focused on the backend/API scope represented in the supplied project source; a React or other frontend application is not included in the supplied project.

3. Business Objectives

- Provide a centralized API for user authentication and file management.
- Allow authenticated users to upload and manage their own files through controlled API operations.
- Allow administrators to review files across users through an administrator-only endpoint.
- Protect file access through JWT authentication, role checks, and ownership validation.
- Store file metadata in SQL Server while keeping file content in Azure Blob Storage.
- Validate uploaded files for supported type, content type, empty content, and maximum size.
- Provide consistent HTTP responses and centralized error handling for common API failures.
- Keep data access and business logic separated through repositories and services.

4. Scope

4.1 In Scope for Initial Scope

- User registration using name, email, and password.
- User login using email and password with JWT token generation.
- User and Admin role handling through JWT role claims.
- File upload using multipart/form-data.
- User gallery listing for files uploaded by the authenticated user.
- Authorized file retrieval by file identifier.

- Replacement of an existing file for the owner or an administrator.
- File download using streamed content from Azure Blob Storage.
- Deletion of file metadata and corresponding blob content.
- Administrator-only listing of all files.
- File metadata persistence through Entity Framework Core and SQL Server.
- File storage through Azure Blob Storage.
- Input validation and custom exception handling.
- Swagger/OpenAPI documentation for development and API testing.

4.2 Out of Scope for Initial Release

- A React, Angular, or other frontend application is not included in the supplied project.
- User profile management endpoints beyond registration and login are not implemented.
- Folder or album management is not implemented.
- Search, tagging, and advanced filtering of files are not implemented.
- Image editing, thumbnail generation, OCR, or media processing is not implemented.
- Automated CI/CD pipelines and cloud hosting configuration are not included in the supplied source.
- Email notifications and password reset workflows are not implemented.

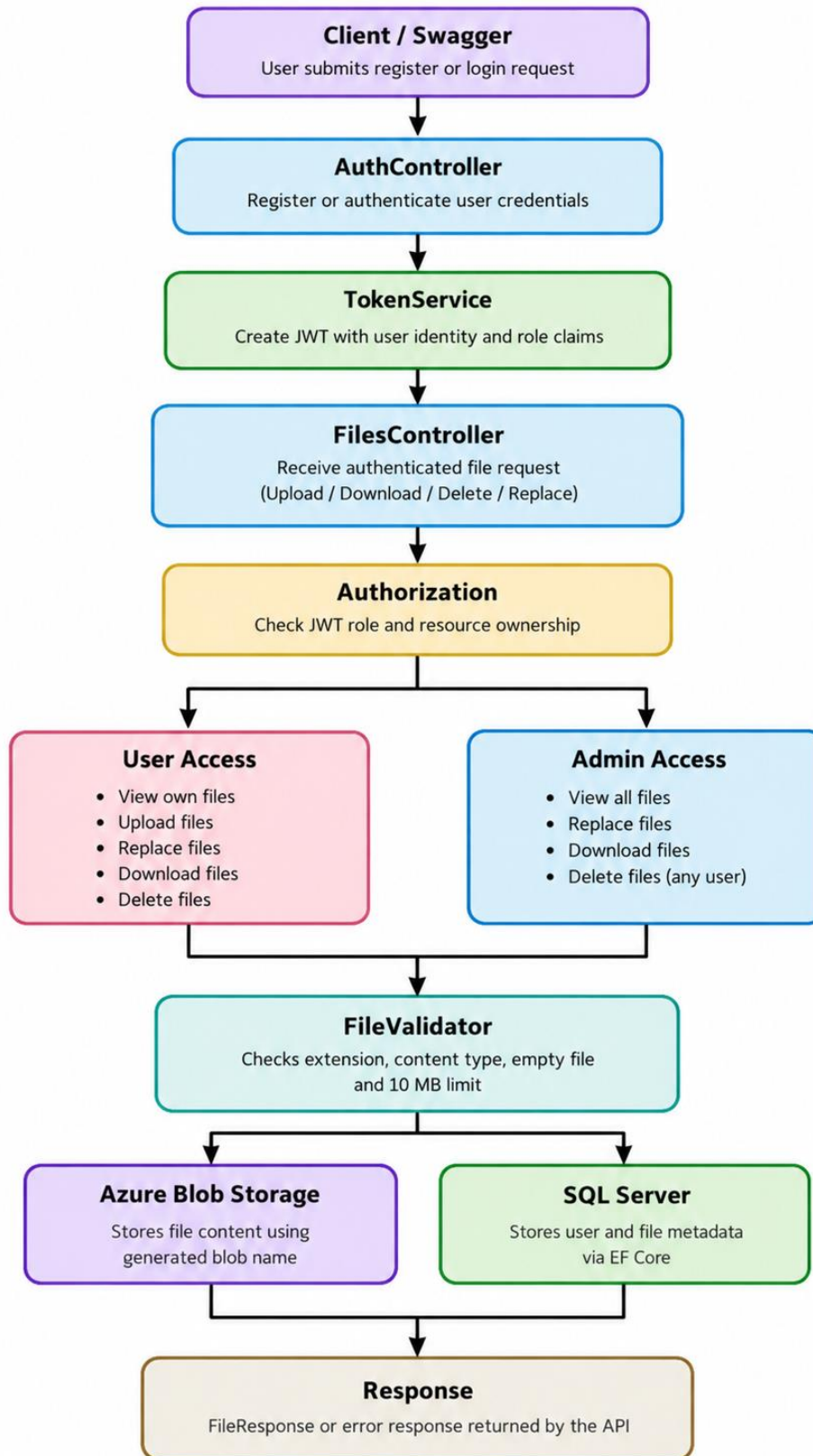
5. Stakeholders and Personas

Persona	Responsibilities / Goals
Admin	Manage and review file records across users. The Admin role can access the administrator file-list endpoint and can act on files when the authorization rules allow it.
User	Register and log in, upload files, view their own gallery, retrieve, replace, download, and delete files that they are authorized to manage.

6. Business Process / Target Workflow

1. A user registers an account or submits login credentials through the authentication API.
2. For a successful login, the server validates the stored BCrypt password hash and creates a JWT containing user identity and role claims.
3. The client sends the JWT with protected file requests.
4. ASP.NET Core authenticates the token and applies endpoint role requirements where configured.
5. The file service checks resource ownership for non-admin users before returning or modifying a file.
6. For uploads and replacements, FileValidator checks the file before it is stored.
7. The file content is uploaded to Azure Blob Storage under a generated file name.
8. The metadata is saved or updated in SQL Server through the repository layer.
9. The API returns a FileResponse or the corresponding HTTP error response.

WinCapture File Workflow



7. Functional Requirements

ID	Requirement	Business Requirement
FR-001	User Registration	The system shall allow a new user to register by providing a name, email, and password.
FR-002	Login	The system shall authenticate a registered user using email and password and return a JWT on success.
FR-003	Password Hashing	The system shall store passwords as BCrypt password hashes rather than plain text.
FR-004	JWT Claims	The generated JWT shall contain the user identifier, name, email, and role claims.
FR-005	Authorization	Protected endpoints shall require authentication, and role-restricted endpoints shall enforce the configured role.
FR-006	Upload File	An authenticated User shall be able to upload a file using multipart/form-data.
FR-007	View Gallery	A User shall be able to retrieve a list of files uploaded by that User.
FR-008	Get File	An authenticated caller shall be able to request a file representation by file identifier, subject to ownership or Admin authorization.

FR-009	Replace File	A User or Admin shall be able to replace an existing authorized file with another supported file.
FR-010	Download File	An authenticated caller shall be able to download an authorized file as a streamed response.
FR-011	Delete File	An authenticated caller shall be able to delete an authorized file.
FR-012	Admin File Listing	An Admin shall be able to retrieve a list of files across users through GET /api/admin/files.
FR-013	File Extension Validation	Uploaded files shall use the supported .jpg, .jpeg, .png, .gif, or .pdf extensions.
FR-014	Content-Type Validation	The uploaded content type shall match the expected MIME type for the file extension.
FR-015	File Size Validation	The system shall reject empty files and files larger than 10 MB.
FR-016	Metadata Persistence	The system shall save original file name, stored file name, content type, size, uploader, and upload time in SQL Server.
FR-017	Blob Storage	The system shall store file content in the configured Azure Blob Storage container.
FR-018	Error Handling	The system shall translate known application exceptions into appropriate

		HTTP status codes and JSON error responses.
FR-019	Swagger/OpenAPI	The development environment shall expose OpenAPI documentation and Swagger UI for API inspection and testing.
FR-020	Secure File Ownership	A non-admin user shall not be allowed to access, replace, download, or delete another user's file.

8. Role-Based Access Requirements

Capability	Admin	User
Register account	No	Yes
Login / Authentication	Yes	Yes
Upload file	No	Yes
View own gallery	Yes	Yes
Get authorized file	Yes	Yes
Replace authorized file	Yes	Yes
Download authorized file	Yes	Yes
Delete authorized file	Yes	Yes
View all files	Yes	No

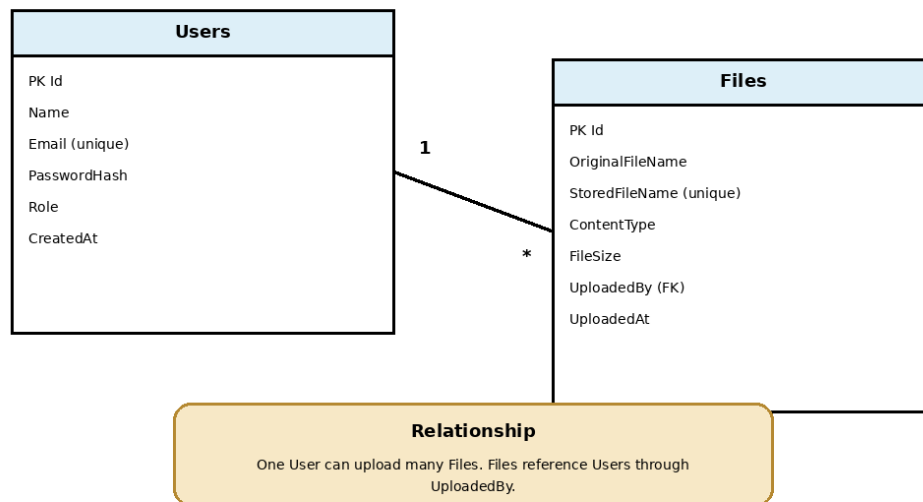
Access /api/admin/files	Yes	No
-------------------------	-----	----

The role matrix reflects the supplied controller and service implementation. File ownership is enforced in FileService for non-admin users, while the AdminController is explicitly protected with the Admin role. File upload itself is restricted at the controller level to the User role.

9. Data Requirements

Entity	Key Data
User	Id, Name, Email, PasswordHash, Role, CreatedAt
FileMetadata	Id, OriginalFileName, StoredFileName, ContentType, FileSize, UploadedBy, UploadedAt

Entity Relationship Diagram



Relationships: One User can have many FileMetadata records. Each file record references its owner through UploadedBy. The relationship is implemented with a foreign key and configured

with restricted delete behavior so that deleting a User does not automatically delete dependent files.

10. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Security	The system shall use JWT bearer authentication, role-based authorization, BCrypt password hashing, ownership checks, and input validation.
NFR-002	Performance	The API should use asynchronous database and storage calls so request processing does not block unnecessarily.
NFR-003	Scalability	The separation of API, database metadata, and blob content allows the system to scale storage independently of SQL metadata.
NFR-004	Availability	The application should provide reliable access when the API, SQL Server, and configured Azure Blob Storage are available.
NFR-005	Maintainability	Controllers, services, repositories, validators, models, DTOs, and middleware shall remain separated into clear responsibilities.
NFR-006	Observability	Unexpected failures shall be logged by the exception middleware with the HTTP method, path, exception, and request identifier.

NFR-007	Usability	Swagger UI shall provide a practical way to inspect and test the backend API during development.
NFR-008	Data Integrity	SQL constraints, unique indexes, foreign keys, and explicit metadata properties shall maintain consistent stored data.
NFR-009	Configuration Safety	The application shall fail fast when required connection strings, JWT settings, or Azure storage settings are missing or invalid.

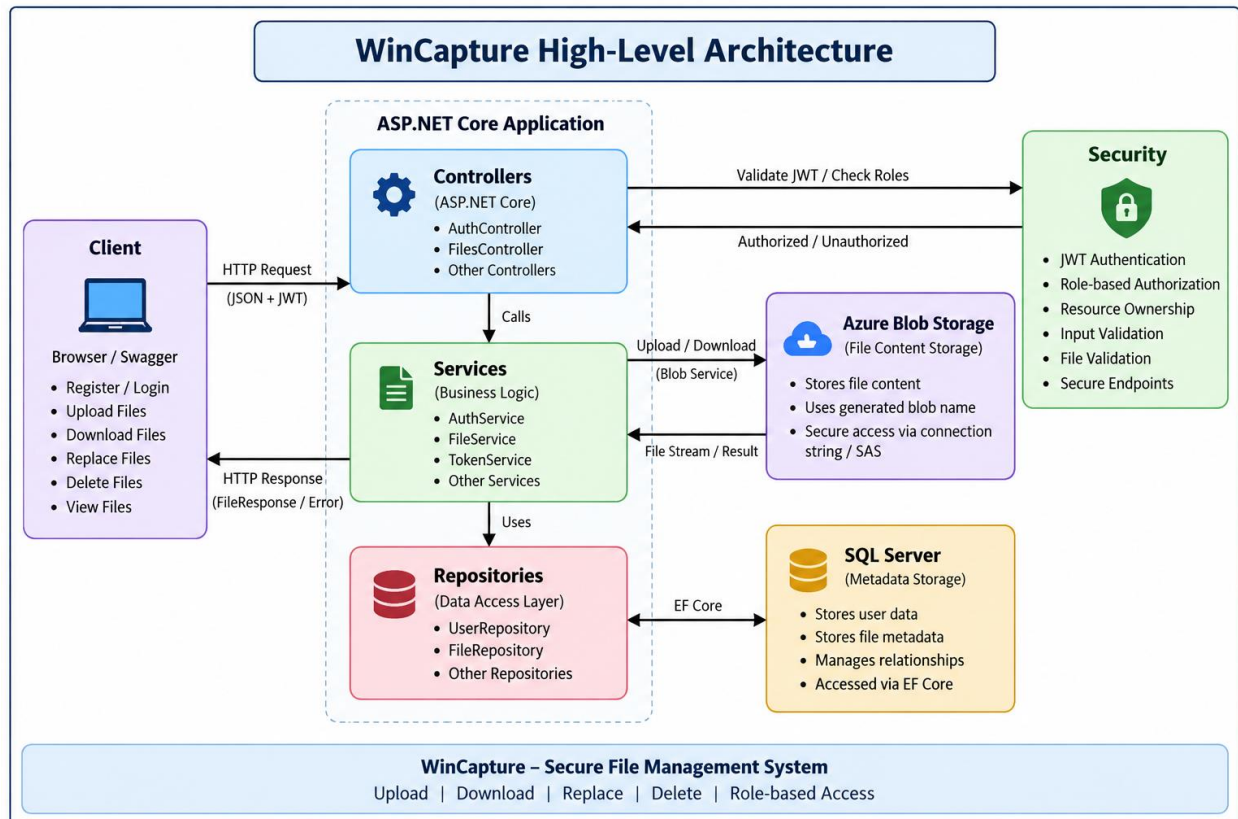
11. Proposed Solution / Technology

Layer	Proposed Technology
Language	C#
Runtime	.NET 10
Backend	ASP.NET Core Web API
ORM	Entity Framework Core
Database	Microsoft SQL Server
Object Storage	Azure Blob Storage
Authentication	JWT Bearer Authentication
Password Security	BCrypt.Net-Next

API Documentation	OpenAPI / Swagger UI
Azure Authentication	Azure.Identity / DefaultAzureCredential

12. High-Level Architecture

The client or Swagger UI sends HTTP requests to the ASP.NET Core application. Controllers expose authentication, file, and administrator operations. Middleware handles exceptions, while JWT authentication and authorization protect the API. Business operations are handled by services, and repositories use Entity Framework Core to persist metadata in SQL Server. File content is sent to Azure Blob Storage through the `IStorageService` abstraction.



13. API Requirements

Area	Representative Endpoints	Authentication / Access
Authentication	POST /api/auth/register; POST /api/auth/login	Anonymous
Files	POST /api/files	User
Gallery	GET /api/files	User / Admin authenticated; service returns current user gallery
File	GET /api/files/{id}	User / Admin with ownership check
Replace	PUT /api/files/{id}	User / Admin with ownership check
Download	GET /api/files/{id}/download	User / Admin with ownership check
Delete	DELETE /api/files/{id}	User / Admin with ownership check
Admin Files	GET /api/admin/files	Admin

The documented API may return 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 413 Payload Too Large, and 500 Internal Server Error depending on the request and application outcome.

14. TDD and Quality Requirements

The supplied project does not contain a dedicated test project in the archive, so specific automated test results cannot be claimed from the source. The quality plan should focus on the implemented authentication, ownership checks, validation rules, storage operations, and API status codes.

- Test successful registration and duplicate email handling.
- Test successful login and invalid credential handling.
- Test JWT validation, expiration, role claims, and protected endpoints.
- Test supported and unsupported file types, MIME mismatches, empty files, and files over 10 MB.
- Test that a User cannot access another user's file.
- Test Admin access to the administrator file listing.
- Test upload, replace, download, and delete behavior with storage failures.
- Test exception middleware and expected HTTP status codes.

15. SDLC Deliverables

SDLC Phase	Expected Deliverables
Requirement Analysis	BRD, functional requirements, non-functional requirements, scope and acceptance criteria
Planning	Task plan, responsibilities, milestone tracking and risk list
Design	High-level architecture, entity relationship diagram, API contract and component responsibilities

Development	ASP.NET Core API, DTOs, models, services, repositories, validation and middleware
Testing	Unit tests for services, repository tests where practical, API and authorization tests
Deployment	Application configuration, SQL Server setup, Azure Blob configuration and environment settings
Demonstration	Authentication flow, file upload, gallery, file operations and Admin listing demonstration
Maintenance	Defect tracking, monitoring, configuration updates and future enhancement backlog

16. Acceptance Criteria

- A User can register successfully with valid details and receives a created response.
- A registered User can log in with valid credentials and receives a JWT response.
- Protected endpoints reject requests without valid authentication.
- A User can upload only supported files within the 10 MB limit.
- A User can view only the files belonging to that User through the gallery and ownership-protected operations.
- An authorized User or Admin can replace, download, and delete an authorized file.
- An Admin can retrieve all file metadata from GET /api/admin/files.
- File metadata is persisted in SQL Server and file content is stored in Azure Blob Storage.
- Known validation, authorization, not-found, payload-size, and storage failures return the intended HTTP status codes.

- Swagger/OpenAPI is available in the development environment for API inspection and testing.

17. Assumptions and Dependencies

- A SQL Server instance and a valid WinCapture database connection string are available for development.
- The Azure Storage account and blob container named in application configuration are available.
- DefaultAzureCredential can obtain credentials in the runtime environment used to access Azure Blob Storage.
- Required JWT settings are supplied through application configuration and the secret key is at least 256 bits.
- The database schema is created using the supplied SQL script; the project does not use EF Core migrations.
- The project is primarily a backend/API implementation; a frontend is outside the supplied source.
- Network connectivity to SQL Server and Azure Blob Storage is available when the application is running.
- Role definitions remain limited to User and Admin unless the design is formally changed.

18. Risks and Mitigations

Risk	Impact	Mitigation
Azure storage unavailable	High	Handle storage exceptions consistently and verify Azure credentials and container configuration before deployment.
Authorization gap	High	Keep ownership checks inside the service layer and test negative access scenarios.
Invalid file content	High	Validate extension, content type, empty content and size before storage.
Database failure	High	Use asynchronous EF Core calls, log unexpected exceptions, and test failure behavior.
Configuration errors	Medium	Fail fast for missing JWT, database and storage settings and validate the JWT key length.
Orphaned blob after metadata failure	Medium	Attempt cleanup of the newly stored blob when metadata persistence fails.
Scope expansion	Medium	Keep the initial release focused on authentication, file management, metadata storage and Admin review.

19. Success Measures

- All agreed authentication and file-management workflows can be demonstrated successfully.

- Users can upload, view, replace, download and delete their own files according to the access rules.
- Administrators can review all stored file metadata through the Admin endpoint.
- Files are stored in Azure Blob Storage and their metadata is retained correctly in SQL Server.
- Invalid file requests are rejected before storage and return predictable API responses.
- Critical authentication and authorization scenarios are covered by tests before release.
- The API can be started successfully with the documented application configuration.

20. Final Business Outcome

The project will deliver a backend File Upload and Gallery Platform that provides authenticated access to file-management operations, protects user data through role and ownership checks, stores file content in Azure Blob Storage, and maintains searchable metadata in SQL Server. The implementation demonstrates a structured ASP.NET Core solution with controllers, services, repositories, validation, centralized exception handling, JWT security, and API documentation.

21. Approval

Role	Name	Signature	Date
Mentor	_____	_____	_____
Student / Developer	_____	_____	_____